Name : Muhammad ilyas

Reg no: L1F24BSAI0078

Section: AD1

Instructor Name: Faria Fawad

AI_LAB(03)_TASKS

Task2:The Multi-Criteria Sorter CLO1

Given a list of tuples representing students (Name, Grade, Age), use a Lambda function with sorted() to sort the list primarily by Grade (descending) and secondarily by Age (ascending).

Note: Take a list of five students.

Source code:

```
while True:
    try:
        size = int(input("Enter number of students you have:\t"))
        break
    except ValueError:
        print("Please enter a valid number (e.g., 5, 6)")

students = []

for i in range(size):
    temp = []
    temp_name = input(f"Please enter name for student {i+1} (e.g., Miksi):\t").strip()

    if not temp_name:
        print("Due to invalid input, system auto-assigned name (unknown)!")
        temp_name = "unknown"

    while True:
        try:
            temp_marks = int(input(f"Enter marks for {temp_name} (e.g., 90):\t"))
            temp_age = int(input(f"Enter age for {temp_name} (e.g., 20):\t"))
            break
        except ValueError:
            print("Invalid input detected. Try again!")

    temp.append(temp_name)
    temp.append(temp_marks)
    temp.append(temp_age)

    students.append(tuple(temp))

print("\n unsorted data printing....")
```

```
print(students)

sorted_students = sorted(students, key=lambda student: (-student[1], student[2]))

print("\nSorted Data (Marks: Descending | Age: Ascending)....")
for s in sorted_students:
    print(s)

    """
we can also used pre defined one list too
students = [
    ("Ali", 85, 20),
    ("Aisha", 92, 19),
    ("Bilal", 92, 21),
    ("Zara", 78, 22),
    ("Omer", 85, 19)
]
"""
```

Output:

Entry:

```
print("\nSorted Data (Marks: Descending | Age: Ascending)....")
for s in sorted_students:
    print(s)
```

```
Enter number of students you have:      5
Please enter name for student 1 (e.g., Miksi):  miksi
Enter marks for miksi (e.g., 90):       99
Enter age for miksi (e.g., 20): 23
Please enter name for student 2 (e.g., Miksi):  ilyas
Enter marks for ilyas (e.g., 90):       100
Enter age for ilyas (e.g., 20): 24
Please enter name for student 3 (e.g., Miksi):  sofi
Enter marks for sofi (e.g., 90):        88
Enter age for sofi (e.g., 20):  34
Please enter name for student 4 (e.g., Miksi):  khuram
Enter marks for khuram (e.g., 90):      88
Enter age for khuram (e.g., 20):        23
Please enter name for student 5 (e.g., Miksi):  ali
Enter marks for ali (e.g., 90): 77
Enter age for ali (e.g., 20):   22
```

Sorted:

```
Enter age for ali (e.g., 20):   22

 unsorted data printing....
[('miksi', 99, 23), ('ilyas', 100, 24), ('sofi', 88, 34), ('khuram', 88, 23), ('ali', 77, 22)]

Sorted Data (Marks: Descending | Age: Ascending)....
('ilyas', 100, 24)
('miksi', 99, 23)
('khuram', 88, 23)
('sofi', 88, 34)
('ali', 77, 22)
```

Task1: Execution Timer Decorator **CLO1**

Create a Decorator named @time_it that measures the time a function takes to execute. Apply this to a function that computes the sum of squares for all numbers up to 10 million.

Note: You can use *time.perf_counter()* for high-precision timing.

Source code:

```
import time
def time_it(func):

    def wrapper(limit):
        start = time.perf_counter()
        answer = func(limit)
        end = time.perf_counter()
        print("seconds taken time is:", end - start)
        return answer
    return wrapper

@time_it
def sum_of_squares(limit):
    total = 0
    for number in range(1, limit + 1):
        total = total + (number * number)
    return total

print("sum of square is: ",sum_of_squares(10000000))
```

Output:

```
    def wrapper(limit):
        start = time.perf_counter()
        answer = func(limit)
        end = time.perf_counter()
        print("seconds taken time is:", end - start)
        return answer
    return wrapper

@time_it
def sum_of_squares(limit):
    total = 0
    for number in range(1, limit + 1):
        total = total + (number * number)
    return total

print("sum of square is: ",sum_of_squares(10000000))
```

```
seconds taken time is: 0.8404765889999908
sum of square is:  333333383333335000000
```

Task3: Memory-Efficient Fibonacci **CLO1**

Write a Generator function that yields the Fibonacci sequence up to

terms. Demonstrate its memory efficiency by "looping" through the first 100 terms without creating a 100-element list.

Source code:

```
def fibonacci_generator(terms):
    a = 0
    b = 1
    count = 0
    while count < terms:
        yield a

        next_number = a + b
        a = b
        b = next_number
        count = count + 1
fib_sequence = fibonacci_generator(100)
i=1
for number in fib_sequence:

  if i==1 or i==2 or i==3 or i%10==0:
    print("term :" ,i ," is:", number)
  i+=1
```

Output:

```
            next_number = a + b
            a = b
            b = next_number
            count = count + 1
fib_sequence = fibonacci_generator(100)
i=1
for number in fib_sequence:

  if i==1 or i==2 or i==3 or i%10==0:
     print("term :" ,i ," is:", number)
  i+=1
```

```
term : 1  is: 0
term : 2  is: 1
term : 3  is: 1
term : 10  is: 34
term : 20  is: 4181
term : 30  is: 514229
term : 40  is: 63245986
term : 50  is: 7778742049
term : 60  is: 956722026041
term : 70  is: 117669030460994
term : 80  is: 14472334024676221
term : 90  is: 1779979416004714189
```

Task4: The Secure File Manager CLO1

Create a Context Manager (either as a class or using @contextmanager) that opens a file, converts all incoming text to uppercase before writing it, and ensures the file is closed even if an exception occurs during writing.

Source code:

```
from contextlib import contextmanager

@contextmanager
def secure_file_manager(filename, mode="w"):
    file = None
    try:
        file = open(filename, mode)

        def write_upper(text):
            if isinstance(text, str):
                file.write(text.upper())
            else:
                raise TypeError("input must be a string please")

        yield write_upper

    finally:
        if file:
```

```
        print("closing file safely...")
        file.close()
try:
    with secure_file_manager("student_records.txt") as write:
        write("miksi got 90 marks\n")
        write("sofi got 95 marks\n")


        raise ValueError("detected a crash during writing!")

except ValueError as e:
    print("the program continued running. caught:", e)
```

Output:

```
    finally:
        if file:
            print("closing file safely...")
            file.close()
try:
    with secure_file_manager("student_records.txt") as write:
        write("miksi got 90 marks\n")
        write("sofi got 95 marks\n")


        raise ValueError("detected a crash during writing!")

except ValueError as e:
    print("the program continued running. caught:", e)
```

```
closing file safely...
the program continued running. caught: detected a crash during writing!
```

Task5: **CLO1**

Answer in your own words:

1. **Question:**

For a Decorator to work, it relies on a functional programming concept called a "closure." Explain how the inner wrapper function retains access to the func argument even after the outer decorator function has finished executing.

1. **Answer:**

A closure happens when an inner function remembers variables from its outer function even after the outer function has finished running. In a decorator, the wrapper function keeps a reference to func because it was defined inside the decorator. Python stores this reference in the closure, so even when the decorator function ends, the wrapper still has access to func and can call it later.

2. Question:

Why are Lambda functions in Python limited to a single expression rather than multiple statements? Contrast this with a standard def function in terms of its return behavior.

2.Answer:

Lambda functions are designed to be small and simple anonymous functions, so they are limited to a single expression and cannot contain multiple statements like loops or assignments. A lambda automatically returns the value of its expression. In contrast, a normal function defined with def can contain many statements, complex logic, and must explicitly use return to send a value back.

3. Question:

In a Generator function, what exactly happens to the local variables and the instruction pointer when the yield keyword is encountered, and how does this differ from a standard return?

3.Answer:

When a generator reaches the yield keyword, it sends the current value to the caller but pauses the function instead of ending it. Python saves the local variables and the current execution position (instruction pointer). When the generator is called again, it resumes from the same point. This is different from return, which completely ends the function and discards its local state.

4. Question:

When using a Context Manager via the with statement, which specific "dunder" (double underscore) method is responsible for handling exceptions that occur within the block, and how can it prevent an exception from propagating?

4.Answer: The method responsible is __exit__(). When an exception occurs n inside the with block, Python calls __exit__(exc_type, exc_value, traceback). If this method returns True, it tells Python that the exception was handled, so the exception will not propagate further. If it returns False (or nothing), the exception continues to propagate normally.